Function	Description
accuracy(fc1, test)	Compute the accuracy of forecasts (stored in the object fc1) from a model with the values in a test set (test)
aov(y ~ x1 + x2 + x2)	Conduct one-way or two-way ANOVA e.g. mod1 <- aov(y ~ x1 + x2)
Arima(x, order = c(p,d,q))	Fit a non-seasonal ARIMA model to a time series dataset x
attach(data)	Attach a dataset (data) to the R workspace so that you can more easily refer to the variables within that dataset
auto.arima(x)	Fit a seasonal or non-seasonal ARIMA model to a time series dataset x where the order of the model is not specified and is determined using an automated algorithm
autolayer()	Add additional graphical elements to a plot (e.g. a title or some forecasts)
autoplot(x)	Plot a time series x
BoxCox(x, lambda = lam)	Variance-transform a time series x, where the value of lambda is obtained using BoxCox.lambda()
BoxCox.lambda(x)	Determine the value of lambda to use when performing a variance-stabilising transformation on a time series x
boxplot(x)	Plots boxplots of variables in numeric dataset x
checkresiduals(mod1)	Check the residuals of a fitted model mod1
cor.test(x, y, method = "spearman", data = my_data)	Conduct Spearman's rank correlation coefficient test on two variables x and y
decompose(x, type = 'multiplicative')	Perform a multiplicative classical decomposition of a time series x
diff(x, lag = p)	Difference a dataset x at lag p. For a first order difference at lag 1, the lag argument does not need to be specified.
friedman.test(y = data$dat, groups = data$group, blocks = data$block)	Perform a Friedman test where 'dat' is the response variable in your data, 'group' the treament variable and 'block' the blocking variable in your data.
forecast(mod1, h = h)	Compute forecasts for a time series model mod1 for a time horizon of h time periods
ggAcf(x)	Plot the ACF of a dataset x
ggPacf(x)	Plot the PACF of a dataset x
ggtitle('title')	Add a title to a ggplot
glm(y ~ x1 + x2 + x3)	A function used to fit generalized linear models. We will use it in this course to fit logistic regression models e.g. mod1 <- glm(y ~ x1 + x2 + x3, family = "binomial")
hist(x)	Plots a histogram of numeric variable x
holt(x)	Fit a Holt's linear trend model to a time series x
hw(x, seasonal = 'multiplicative')	Fit a Holt-Winter's seasonal model to a time series x
install.packages("packagename")	Install the package "packagename". Note you will need to be connected to the internet for this to work
interaction.plot(x.factor = data$factorA, trace.factor = data$factorB,response = data$response)	Creates an interaction plot to visualise how the mean of response changes across levels of factorA, with separate lines (traces) for each level of factorB. This helps to assess possible interaction effects between the two factors.
kruskal.test(y ~ x, data = my_data)	Perform a Kruskal-Wallis test where y is the response variable in your data and 'group' the treatment variable in your data (my_data)
library(packagename)	Loads all of the content of package "packagename" into your workspace so that it is available for you to use
lm(y ~ x1 + x2 + x3)	Fits a multiple linear regression model of y as a function of variables x1, x2 and x3
lm(y ~ x1:x3)	Fits a multiple linear regression model of y as a function of variables x1, x2 and x3 and all their interaction terms
ma(x,k)	Apply a moving average smoothing model of order k to a dataset x
max(x)	Finds the maximum value of a numeric variable x
meanf(x, h = h)	Apply the mean (or average) simple forecasting model to a time series dataset x for a forecast horizon of h time periods
median(x)	Finds the median value of a numeric variable x
min(x)	Finds the minimum value of a numeric variable x
model.tables(model_object, type ="effects", se = TRUE)	Find estimated effects and associated standard errors, replace "effects" with "means" to get the estimated treatment means.
mod1 <- lm(y ~ x1)	Computes a simple linear regression with y the dependent numeric variable and x1 the independent numeric variable.'lm' stands for 'linear model'. Saves the results of the fitted model to the object "mod1"
naive(x, h = h)	Apply the naïve forecasting model to a time series dataset x for a forecast horizon of h time periods
names(dataset)	Returns the names of all the variables in a dataset
ndiffs(x)	Determine the number of rounds of differencing needed to make a time series x stationary
nsdiffs(x)	Determine the number of rounds of differencing at the seasonal lag needed to make a time series x stationary
plot()	Generic plotting function. Often used to plot a scatterplot of two numeric variables e.g. plot(x,y). Can also be used on a model object to plot diagnostic plots e.g. plot(mod1) where mod1 is an object created to store the results of fitting a regression model
residuals(mod1)	Obtain the residuals from a fitted model object mod1
rwf(y,h = h, drift = TRUE)	Apply the drift simple forecasting model to a time series dataset x for a forecast horizon of h time periods
sd(x)	Compute the standard deviation of a dataset x
ses(x)	Apply a simple exponential smoothing model to a time series x
snaive(x, h = h)	Apply the seasonal naïve simple forecasting model to a time series dataset x for a forecast horizon of h time periods
step(fit2,direction="forward", scope=list(upper=fit1,lower=fit2))	Perform stepwise regression e.g. For forward selection in a dataset called fresh. fit1 <- lm(demand ~ ., data=fresh); fit2 <- lm(demand ~ 1, data=fresh); step(fit2,direction="forward", scope=list(upper=fit1,lower=fit2))
str(object)	Displays the structure of an object
summary(mod1)	Specific application of the summary function to print out a summary of a fitted model mod1
summary(x)	Generic function that produces a six-number summary of object x (or all variables within a dataset x)
table(x)	Determine how many observations of each category are present in a variable or dataset that consists of categorical data
tapply(data$response, data$treatment, function_name)	Applies function_name (e.g., mean, sd) to the values in response, grouped according to the levels of treatment. This returns the function result for each group separately.
ts(x, start = (a,b), frequency = f)	Create a time series from a variable x that starts in period (a,b) where a generally refers to a year and b a period within that year and frequency refers to how often observations were recorded in each year
which.max(x)	Determine the index of the maximum value in a dataset x
wilcox.test(x,y,alternative = 'greater', paired = TRUE)	Perform a one-sided greater than Wilcoxon Signed Rank sum test between the variables x and y. The arguments 'two.sided' or 'less' can also be used for alternative, depending on the type of test you are conducting.
wilcox.test(x,y,alternative = 'greater')	Perform a one-sided greater than Wilcoxon Rank sum/Mann-Whitney test between the variables x and y. The arguments 'two.sided' or 'less' can also be used for alternative, depending on the type of test you are conducting.
window(x, start = c(a,b), end = (c,d))	Subset a time series x, starting a time period (a,b) and ending at time period (c,d) where a and c generally refer to years and b and d to periods within those years
xlab('xlab')	Add a x-axis label to a plot
ylab('ylab')	Add a y-axis label to a plot

Package	Description
forecast	Automatic Time Series Forecasting
fpp2 or fpp3	Datasets from Forecasting Principles and Practice
ggplot2	Grammar of graphics